



EAST WEST UNIVERSITY

Assignment

Submitted to:

- ***Dr. Md. Manowarul Islam***
- ***Adjunct faculty***
- ***Department of Computer Science and Technology***
- ***East West University***

Submitted by:

- ***Name*** : ***Rakib Al Hasan***
- ***Student ID*** : ***2024-1-60-143***
- ***Course Title*** : ***Data Structure and Algorithm***
- ***Course Code*** : ***CSE 207***
- ***Section*** : ***10***
- ***Semester*** : ***Summer 2025***
- ***Date of Submission*** : ***30.08.2025***

1. Write a program in to insert a new node at the beginning of a linked list and display the Linked list

Sample Input	Sample Output
Input element: 10	10
Input element: 20	20 10
Input element: 30	30 20 10

```
#include<stdio.h>
#include<stdlib.h>

struct Node
{
    int data;
    struct Node *link;
};

struct Node *head = NULL;

void insertAtBeginning(int item)
{
    struct Node *node = (struct Node*) malloc(sizeof(struct Node));
    if(node == NULL)
    {
        printf("\tMemory allocation failed\n");
        return;
    }
    node->data = item;
    node->link = head;
    head = node;
}

void display()
{
    struct Node *temp = head;
    printf("\n\tLink list contains : ");
    while(temp != NULL)
    {
```

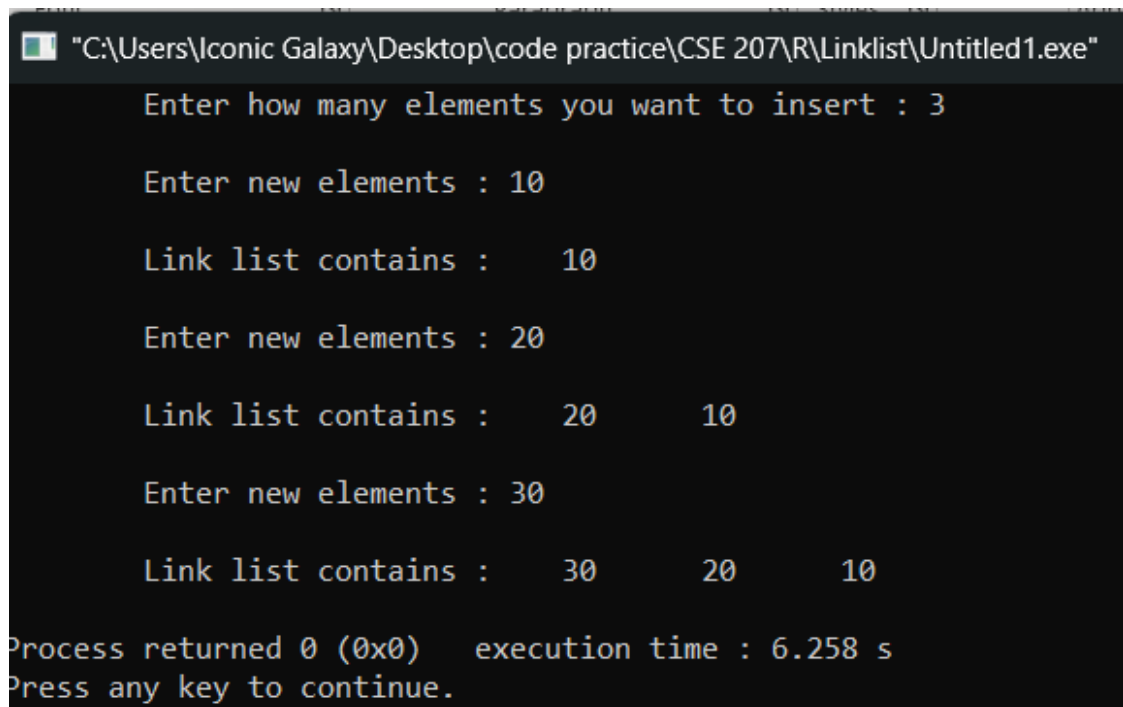
```

        printf("\t%d", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

int main()
{
    int n, item;
    printf("\tEnter how many elements you want to insert : ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
    {
        printf("\n\tEnter new elements : ");
        scanf("%d", &item);
        insertAtBeginning(item);
        display();
    }
    return 0;
}

```



The screenshot shows a Windows command prompt window titled "C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Linklist\Untitled1.exe". The program prompts the user to enter the number of elements to insert (3). It then iteratively prompts for new elements (10, 20, 30) and displays the current state of the linked list after each insertion. The output shows the list growing from [10] to [20, 10] to [30, 20, 10]. At the bottom, it states "Process returned 0 (0x0) execution time : 6.258 s" and "Press any key to continue."

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Linklist\Untitled1.exe"
Enter how many elements you want to insert : 3

Enter new elements : 10

Link list contains :    10

Enter new elements : 20

Link list contains :    20    10

Enter new elements : 30

Link list contains :    30    20    10

Process returned 0 (0x0)   execution time : 6.258 s
Press any key to continue.

```

- Write a program in C to find a specific element of a singly linked list. If not found then insert that key value at the middle of the Linked list (Create a linked list first. Insert at the end of the linked list for creating)

Sample Input	Sample Output
Linked List elements: 10 20 30 40 50 Search key: 40	Element is found
Linked List elements: 10 20 30 40 50 Search key: 70	Not Found 70 Inserted at the middle 10 20 70 30 40 50

```
#include<stdio.h>
#include<stdlib.h>
```

```
struct Node
{
    int data;
    struct Node *link;
};
```

```
struct Node *head = NULL;
```

```
void insertAtEnd(int item)
{
    struct Node *node = (struct Node*) malloc(sizeof(struct Node));
    node->data = item;
    node->link = NULL;
    if(head == NULL)
    {
        head = node;
    }
    else
    {
        struct Node *temp = head;
        while(temp->link != NULL) temp = temp->link;
        temp->link = node;
    }
}
```

```

int countNodes()
{
    int count = 0;
    struct Node *temp = head;
    while(temp != NULL)
    {
        count++;
        temp = temp->link;
    }
    return count;
}

void insertAtMiddle(int item)
{
    int pos = countNodes() / 2;
    struct Node *node = (struct Node*) malloc(sizeof(struct Node));
    node->data = item;

    struct Node *temp = head;
    for(int i = 1; i < pos; i++) temp = temp->link;
    node->link = temp->link;
    temp->link = node;
}

int search(int key)
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        if(temp->data == key)
        {
            return 1;
        }
        temp = temp->link;
    }
    return 0;
}

void display()
{
    struct Node *temp = head;
    while(temp != NULL)

```

```

    {
        printf("\t%d", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

int main()
{
    int n, item, key;
    printf("\tEnter how many elements you want to insert : ");
    scanf("%d", &n);

    printf("\n\tEnter the elements : ");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &item);
        insertAtEnd(item);
    }

    printf("\n\tLinked List elements : ");
    display();

    printf("\n\tEnter search key : ");
    scanf("%d", &key);

    if(search(key))
    {
        printf("\n\tElement is found\n");
    }
    else
    {
        printf("\n\tNot Found\n");
        insertAtMiddle(key);
        printf("\t%d Inserted at the middle\n", key);
    }

    printf("\n\tLinked List elements : ");
    display();

    return 0;
}

```

```
"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Linklist\Untitled2.exe"
Enter how many elements you want to insert : 5
Enter the elements : 10
20
30
40
50

Linked List elements : 10      20      30      40      50

Enter search key : 40

Element is found

Linked List elements : 10      20      30      40      50

Enter search key : 70

Not Found
70 Inserted at the middle

Linked List elements : 10      20      70      30      40      50
```

3. Write a program in C to delete from the beginning of a Linked List. (Create the Linked list first)

Sample Input	Sample Output
Linked List elements: 10 20 30 40 50	20 30 40 50

```
#include<stdio.h>
#include<stdlib.h>
struct Node
{
    int data;
    struct Node *link;
};
struct Node *head = NULL;
void insertAtEnd(int item)
{
    struct Node *node = (struct Node*) malloc(sizeof(struct Node));
    node->data = item;
    node->link = NULL;
    if(head == NULL)
    {
        head = node;
    }
    else
    {
        struct Node *temp = head;
        while(temp->link != NULL) temp = temp->link;
        temp->link = node;
    }
}
```

```

}

void deleteFromBeginning()
{
    if(head == NULL)
    {
        printf("\nList is empty!!\n");
        return;
    }
    struct Node *temp = head;
    head = head->link;
    free(temp);
}

void display()
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("\t%d", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

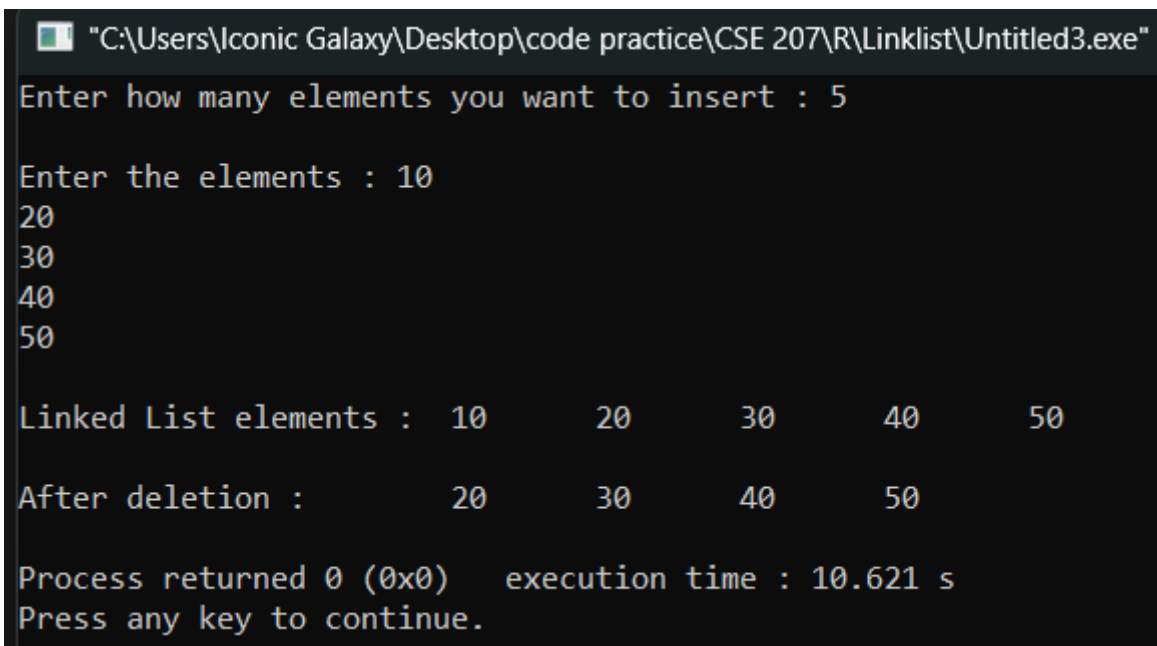
int main()
{
    int n, item;
    printf("Enter how many elements you want to insert : ");
    scanf("%d", &n);

```

```

printf("\nEnter the elements : ");
for(int i = 0; i < n; i++)
{
    scanf("%d", &item);
    insertAtEnd(item);
}
printf("\nLinked List elements : ");
display();
deleteFromBeginning();
printf("\nAfter deletion : ");
display();
return 0;
}

```



```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Linklist\Untitled3.exe"
Enter how many elements you want to insert : 5

Enter the elements : 10
20
30
40
50

Linked List elements : 10      20      30      40      50

After deletion :      20      30      40      50

Process returned 0 (0x0)   execution time : 10.621 s
Press any key to continue.

```

4. Write a program in C to delete the first occurrence of a specific element from a linked list. (Create the Linked list first)

Sample Input	Sample Output
Linked List elements: 10 20 30 20 50 60 Key: 20	10 30 20 50 60

```
#include<stdio.h>
#include<stdlib.h>

struct Node
{
    int data;
    struct Node *link;
};

struct Node *head = NULL;

void insertAtEnd(int item)
{
    struct Node *node = (struct Node*) malloc(sizeof(struct Node));
    node->data = item;
    node->link = NULL;
    if(head == NULL)
    {
        head = node;
    }
    else
    {
        struct Node *temp = head;
        while(temp->link != NULL) temp = temp->link;
        temp->link = node;
    }
}

int deleteKey(int key)
{
    struct Node *temp = head, *prev = NULL;
```

```

while(temp != NULL && temp->data != key)
{
    prev = temp;
    temp = temp->link;
}

if(temp == NULL)
{
    return 0;
}

if(prev == NULL)
{
    head = head->link;
}
else
{
    prev->link = temp->link;
}
free(temp);
return 1;
}

void display()
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("%d\t", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

int main()
{
    int n, item, key;
    printf("Enter how many elements you want to insert : ");
    scanf("%d", &n);

    printf("\nEnter the elements : ");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &item);
    }
}

```

```

        insertAtEnd(item);
    }

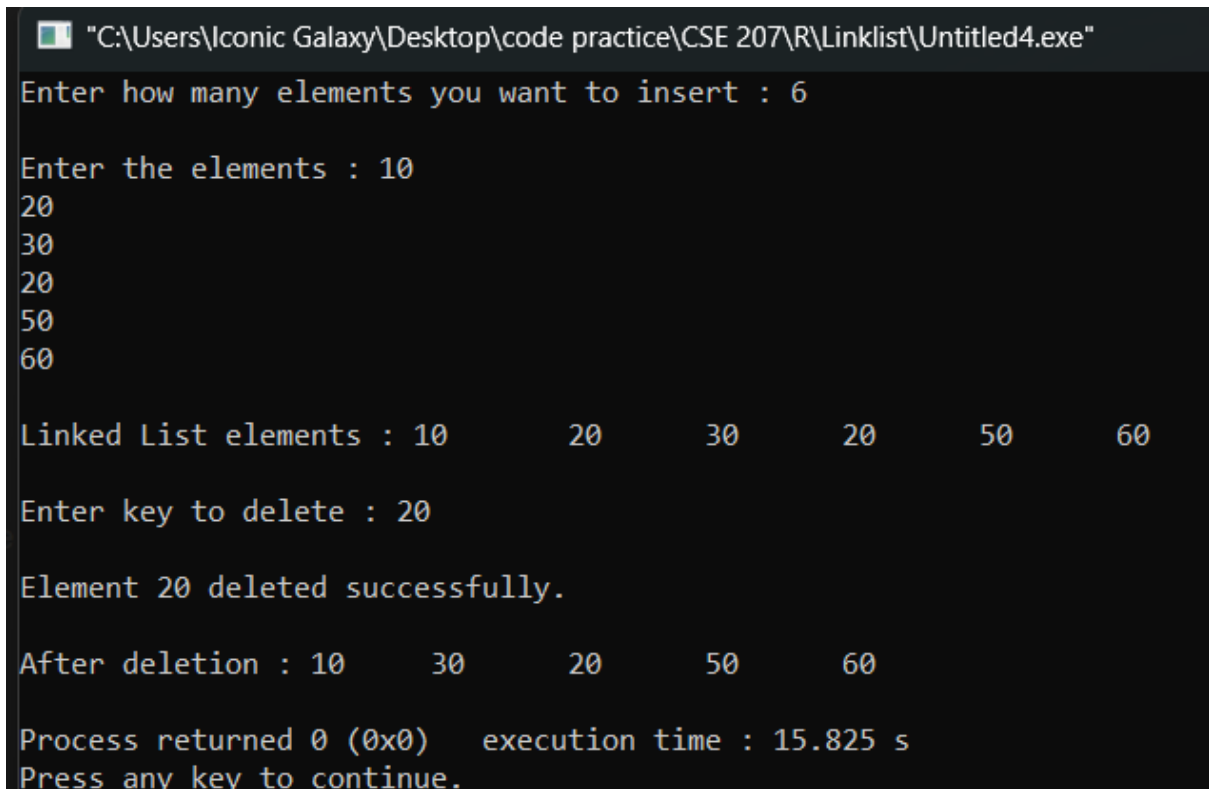
    printf("\nLinked List elements : ");
    display();

    printf("\nEnter key to delete : ");
    scanf("%d", &key);

    if(deleteKey(key))
    {
        printf("\nElement %d deleted successfully.\n", key);
        printf("\nAfter deletion : ");
        display();
    }
    else
    {
        printf("\nElement %d not found. Deletion not possible.\n", key);
    }

    return 0;
}

```



```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Linklist\Untitled4.exe"
Enter how many elements you want to insert : 6

Enter the elements : 10
20
30
20
50
60

Linked List elements : 10      20      30      20      50      60

Enter key to delete : 20

Element 20 deleted successfully.

After deletion : 10      30      20      50      60

Process returned 0 (0x0)   execution time : 15.825 s
Press any key to continue.

```

5. Write a program in C to reverse a linked list. (Create the Linked list first)

Sample Input	Sample Output
Linked List elements: 10 20 30 40 50	 50 40 30 20 10

```
#include<stdio.h>
#include<stdlib.h>

struct Node
{
    int data;
    struct Node *link;
};

struct Node *head = NULL;

void insertAtEnd(int item)
{
    struct Node *node = (struct Node*) malloc(sizeof(struct Node));
    node->data = item;
    node->link = NULL;
    if(head == NULL)
    {
        head = node;
    }
    else
    {
        struct Node *temp = head;
        while(temp->link != NULL) temp = temp->link;
        temp->link = node;
    }
}

void reverseList()
{
    struct Node *prev = NULL, *curr = head, *next = NULL;
    while(curr != NULL)
    {
        next = curr->link;
        curr->link = prev;
        prev = curr;
        curr = next;
    }
    head = prev;
}
```

```

}

void display()
{
    struct Node *temp = head;
    while(temp != NULL)
    {
        printf("\t%d", temp->data);
        temp = temp->link;
    }
    printf("\n");
}

int main()
{
    int n, item;
    printf("Enter how many elements you want to insert : ");
    scanf("%d", &n);

    printf("Enter the elements : ");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &item);
        insertAtEnd(item);
    }

    printf("\nLinked List elements : ");
    display();

    reverseList();

    printf("\nReversed Linked List : ");
    display();

    return 0;
}

```

```

Enter how many elements you want to insert : 5
Enter the elements : 10
20
30
40
50

Linked List elements : 10 20 30 40 50

Reversed Linked List : 50 40 30 20 10

Process returned 0 (0x0)   execution time : 9.400 s

```